

Zameen Scraper Evaluation Report

GitHub Repository:

[View the Original & Refactored Source Code here](#)

1. Modularity

Are the modules highly cohesive? (Each module has a single, well-defined purpose)

Before Refactoring:

- **PASS**

After Refactoring:

- **PASS**

Original code: Mostly followed, but main did too much heavy lifting. Refactored code: Classes clearly define responsibilities.

Is the coupling between modules low? (Minimal and explicit dependencies between them)

Before Refactoring:

- **FAIL**

After Refactoring:

- **PASS**

Original code: Tightly coupled to shared state (lists/locks). Refactored code: ZameenScraper instance encapsulates all shared threading state.

Does each module hide its internal implementation details? (Information

Hiding/Encapsulation)

Before Refactoring:

- **PASS**

After Refactoring:

- **PASS**

Original code: Functions handled their specific domains well. Refactored code: Classes further improve encapsulation.

Are interfaces between modules well-defined, simple, and stable?

Before Refactoring:

- **PASS**

After Refactoring:

- **PASS**

Both versions maintain clear, typed function/method signatures.

Can individual modules be tested independently of the rest of the system?

Before Refactoring:

- **FAIL**

After Refactoring:

- **PASS**

Original code: Extraction functions made live HTTP requests internally. Refactored code: ZameenParser operates purely on string/soup inputs, making it fully unit-testable offline.

Can you replace or upgrade one module without needing to modify others?

Before Refactoring:

- **PASS**

After Refactoring:

- **PASS**

Both versions allow swapping specific extraction logic easily.

2. Readability

Are variable, function, and class names descriptive, clear, and intention-revealing?

Before Refactoring:

- **PASS**

After Refactoring:

- **PASS**

Both versions use highly descriptive names.

Is the code formatting consistent according to standard style guides?

Before Refactoring:

- **PASS**

After Refactoring:

- **PASS**

Both versions respect standard PEP 8 practices.

Are complex expressions broken down into smaller, well-named intermediate variables or functions?

Before Refactoring:

- **PASS**

After Refactoring:

- **PASS**

Both versions successfully break down complex dictionary navigations.

Are comments used to explain *why* a decision was made, rather than restating *what* the code does?

Before Refactoring:

- **FAIL**

After Refactoring:

- **PASS**

Original code: Comments merely restated obvious code actions. Refactored code: Comments explain reasons behind design decisions (e.g., impersonating Chrome to bypass WAF).

Are functions short, fitting on a single screen, and focused on a single task?

Before Refactoring:

- **FAIL**

After Refactoring:

- **PASS**

Original code: Functions were over 70 lines long doing multiple things. Refactored code: Broken down into `_extract_location`, `_extract_basic_info`, etc.

Is the logic flow straightforward, avoiding deep nesting and overly complex conditionals?

Before Refactoring:

- **FAIL**

After Refactoring:

- **PASS**

Original code: Massive "arrow code" nesting in the crawler loop. Refactored code: Flattened by extracting the page processing logic into `_process_search_page`.

Are standard idioms and patterns of the programming language followed?

Before Refactoring:

- **PASS**

After Refactoring:

- **PASS**

Both versions follow standard idioms.

3. Maintainability

Is the codebase well-documented at a high level? (Architecture overview, README)

Before Refactoring:

- **FAIL**

After Refactoring:

- **PASS**

Original code: No high-level documentation. Refactored code: Classes include descriptive docstrings explaining their architectural role.

Are there automated tests (unit, integration) that verify functionality and catch regressions quickly?

Before Refactoring:

- **FAIL**

After Refactoring:

- **FAIL**

No tests are provided in either version (though the refactored code makes it possible).

Is the code structured to minimize the "blast radius" of changes?

Before Refactoring:

- **PASS**

After Refactoring:

- **PASS**

Changes in extraction logic do not break the crawling orchestrator in either version.

Are magic numbers and hardcoded strings extracted into named constants or configuration files?

Before Refactoring:

- **FAIL**

After Refactoring:

- **PASS**

Original code: Inline timeouts and retry limits. Refactored code: Moved to uppercase constants at the top of the file.

Is duplicated logic avoided by extracting common functionality? (DRY)

Before Refactoring:

- **FAIL**

After Refactoring:

- **PASS**

Original code: Network sessions initialized repeatedly. Refactored code: Handled centrally.

Are error handling and logging implemented robustly and consistently across the system?

Before Refactoring:

- **FAIL**

After Refactoring:

- **PASS**

Original code: Raw terminal print statements with manual ANSI codes. Refactored code: Standard Python logging module utilized.

Does the design follow established object-oriented or functional design principles? (e.g., SOLID)

Before Refactoring:

- **FAIL**

After Refactoring:

- **PASS**

Original code: Procedural, passing global locks around. Refactored code: Object-Oriented, separating Parsing from Scraping logic.